Zur Realisierung des Projekts werden hauptsächlich vier eigene Komponenten verwendet: die Navigator-Node navigator.py, der Partikelfilter mcl.py, die Kartenkonfiguration map_config.yaml und der Micro-ROS-Sketch auf dem Teensy. Zusätzlich laufen der bereitgestellte RPLidar-Treiber (rplidar_ros), der IMU-Node (imu_bno055) und der Micro-ROS Agent. Ziel des Systems ist ein Roboter, der zehn vorgegebene Punkte in einem Labyrinth autonom nacheinander anfährt, an jedem Punkt kurz anhält und sich dabei mit Monte Carlo Localization aus Odometrie, Lidar und IMU lokalisiert.

Die Architektur behält die bewährte Aufteilung aus dem Rennstrecken-Projekt bei: Der Raspberry Pi übernimmt die rechenintensive Arbeit (Partikelfilter, Lidar-Auswertung, Zustandsmaschine), der Teensy die zeitkritische Ansteuerung der beiden Servos und die Radodometrie über die Servo-Feedback-Pins. Neu ist die Kommunikation: Statt eines reinen Int32-Steuerwerts sendet der Pi jetzt Klartext-Kommandos auf dem Topic teensy_cmd, und der Teensy meldet auf odom_feedback die tatsächlich gefahrenen Ist-Werte zurück. Damit wird aus der einseitigen Steuerung ein geschlossener Regelkreis über beide Rechner.

```
R:<dist_m>   kontinuierlich fahren bis Odometrie-Ziel, Feedback D:<ist>
T:<grad>     Drehung auf der Stelle,                   Feedback T:<ist>
D:<dist_m>   blockierende Geradeausfahrt,              Feedback D:<ist>
S:0          sofortiger Stopp,                         Feedback D:<ist>
C:<-100..100> Lenk-Korrektur waehrend der Fahrt (ohne Feedback)
```

Das alte Topic micro_ros_fabinal_subscriber bleibt als Not-Stopp erhalten: Der Wert 500 hält beide Motoren unabhängig vom restlichen Systemzustand an.

Karte und Waypoints

Das Koordinatensystem hat seinen Ursprung in der Startpose: x zeigt in die anfängliche Fahrtrichtung, y nach links, Winkel sind positiv gegen den Uhrzeigersinn. Alle elf Punkte (P1 ist der Start und zählt nicht zur Wertung) und alle Wände stehen in der zentralen Konfigurationsdatei map_config.yaml, sodass nichts davon im Code hartkodiert ist.

```
waypoints:
  - {id: 1,  x: 0.00,  y:  0.00, theta:   0}
  - {id: 2,  x: 2.61,  y:  0.00, theta:   0}
  - {id: 3,  x: 2.22,  y: -0.96, theta: 180}
  ...
walls:
  - {x1: -0.170, y1:  0.270, x2: 3.010, y2:  0.270}  # obere_wand
  - {x1: -0.140, y1: -1.280, x2: 4.650, y2: -1.280}  # untere_wand
```

Die acht Wandsegmente wurden nicht aus einem Bauplan übernommen, sondern aus Referenz-Lidarscans abgeleitet, die vor dem Lauf an den elf Punkten aufgenommen wurden (das manuelle Positionieren zur Vorbereitung ist laut Aufgabenstellung erlaubt). Die Karte beschreibt damit genau das, was der Lidar später auch wirklich sieht, inklusive der realen Abstände zwischen den Wänden.

Final Project: Waypoint-Navigation mit Monte Carlo Localization (Micro-ROS)

Fabian Alexander Feith
Inalbek Akiev
Fakultät 10 | Robotik | Prof. Dr. Algorri

Monte Carlo Localization (mcl.py)

Der Partikelfilter ist vollständig mit numpy vektorisiert und arbeitet mit N = 800 Partikeln, die jeweils eine Pose (x, y, theta) tragen. Beim Start und nach jedem bestätigten Waypoint wird die Wolke eng um die bekannte Pose initialisiert (Streuung 5 cm bzw. 0,05 rad), da es sich um kontinuierliche Lokalisierung mit bekannter Startpose handelt, nicht um das Kidnapped-Robot-Problem.

Das Bewegungsmodell wurde nicht geschätzt, sondern gemessen: Aus acht Kalibrierfahrten über je 2 m ergab sich ein Längsfehler von 0,96 %/m, eine seitliche Streuung von 2,17 %/m und ein systematischer Linksdrift von +1,81 cm/m. Genau diese drei Werte stehen in der Konfiguration (noise_trans, noise_lat, bias_lat) und werden pro Partikel als individuelles Gauß-Rauschen im Roboter-Koordinatensystem aufgeprägt, der Drift als Mittelwertverschiebung des Querfehlers:

```
e = np.random.normal(0, sigma_x, N)                       # Laengsfehler
f = np.random.normal(self.bias_lat * delta_dist, sigma_y, N)  # Quer inkl. Drift
g = np.random.normal(0, sigma_rot, N)
theta = self.particles[:, 2]
self.particles[:, 0] += (delta_dist + e) * np.cos(theta) - f * np.sin(theta)
self.particles[:, 1] += (delta_dist + e) * np.sin(theta) + f * np.cos(theta)
self.particles[:, 2] += delta_theta + g
```

Für das Messupdate wird der Lidar-Scan zunächst auf gültige Werte gefiltert und gleichmäßig auf n_rays = 36 Strahlen heruntergerechnet. 36 Strahlen reichen für die Gewichtung völlig aus und halten die Rechenzeit auf dem Pi klein, denn für jeden Strahl und jedes der 800 Partikel muss die erwartete Distanz per Raycasting gegen alle Wandsegmente berechnet werden. Der Schnittpunkt von Strahl und Wandsegment (A, B) folgt der Geradenschnitt-Formel aus der Vorlesung; ein Treffer zählt nur, wenn er innerhalb des Segments liegt (0 <= t <= 1) und vor dem Roboter (m > 0), bei mehreren Treffern gewinnt der kürzeste:

```
denom = dBx[:, None] * sin_a[None, :] - dBy[:, None] * cos_a[None, :]
t = ((Ay[:, None] - py) * cos_a - (Ax[:, None] - px) * sin_a) / denom
m = ((Ay[:, None] - py) * dBx  - (Ax[:, None] - px) * dBy)  / denom
hit = valid_denom & (t >= 0.0) & (t <= 1.0) & (m > 0.0)
return np.min(np.where(hit, m, np.inf), axis=0)
```

Der Vergleich zwischen gemessener und erwarteter Distanz passiert im Log-Raum. Das ist der entscheidende Unterschied zu einer naiven Multiplikation vieler kleiner Gauß-Werte: Bei 36 Strahlen würden die Produkte numerisch gegen Null laufen (Underflow) und alle Gewichte gleichzeitig sterben. Stattdessen werden die Log-Likelihoods summiert, das Maximum abgezogen und erst dann exponenziert. Ein Likelihood-Floor von 0,01 verhindert zusätzlich, dass einzelne Ausreißer-Messungen ein eigentlich gutes Partikel komplett vernichten (robuste Likelihood mit schweren Rändern, wie in der Vorlesung empfohlen). Strahlen, für

die der Raycast keine Wand findet, bekommen eine feste Straf-Distanz von 10 m statt ignoriert zu werden.

```
diff = z_exp_v - z_meas_v[None, :]
log_like = -0.5 * (diff / self.sigma) ** 2
log_w = np.sum(log_like, axis=1)
log_w -= np.max(log_w)                      # numerisch stabil
new_weights = np.maximum(np.exp(log_w), self.floor)
```

Als Lidar-Standardabweichung ist sigma = 1 cm konfiguriert, passend zur gemessenen Wiederholgenauigkeit des RPLidar A1 auf kurzen Distanzen. Der Einbauversatz des Lidars gegenüber dem Radmittelpunkt (3 cm nach hinten, 0,5 cm nach links) wird vor dem Raycasting auf jede Partikelpose gerechnet, damit Karte und Messung denselben Bezugspunkt haben.

Nach jedem Update folgt systematisches Resampling: ein einziger Zufalls-Startwert, danach N gleichmäßig verteilte Positionen auf der kumulativen Gewichtsverteilung. Das streut weniger als das multinomiale Ziehen aus dem Praktikums-Beispielcode und läuft in O(N) statt O(N**2):

```
cumsum = np.cumsum(self.weights)
start = np.random.uniform(0, 1.0 / self.N)
positions = start + np.arange(self.N) / self.N
indices = np.searchsorted(cumsum, positions)
self.particles = self.particles[indices].copy()
```

Die Pose-Schätzung ist der gewichtete Mittelwert aller Partikel, wobei theta wegen des Winkel-Wraparounds über atan2(mean(sin), mean(cos)) gemittelt wird. Als Gesundheitsindikator wird nach jedem Update die effektive Partikelzahl N_eff = 1/sum(w**2) geloggt; fällt sie stark ab, degeneriert der Filter.

Ein Theta-Update aus dem IMU-Yaw (imu_update mit bewusst großem sigma von 15 Grad) ist implementiert, wurde im finalen Lauf aber deaktiviert: Der BNO055 lief ohne Magnetometer-Stützung und driftete im Yaw zu stark, um als absolute Referenz zu taugen. Die IMU wird stattdessen dort eingesetzt, wo sie zuverlässig ist, nämlich als kurzfristige Heading-Referenz zwischen zwei Fahrsegmenten (siehe Wandkorrektur).

Teensy: Fahr-Primitive und Odometrie

Der Teensy misst die Radwinkel über die PWM-Feedback-Leitungen der beiden Servo360: Aus High- und Low-Zeit wird der Duty-Cycle bestimmt und daraus die absolute Radstellung von 0 bis 359 Grad. Die Differenz zweier Messungen (mit Wraparound-Korrektur) wird aufsummiert und über den Radumfang (d = 0,068 m) in Strecke umgerechnet:

```
dutyCycleRight = ((double)highRight / (double)cycleTimeRight) * 100.0;
thetaRight = 359.0 - ((dutyCycleRight - 2.9) * 360.0) / (97.1 - 2.9 + 1.0);
double diff = thetaRight - thetaRightPrev;
```

```
if (diff < -180.0) diff += 360.0;
if (diff >  180.0) diff -= 360.0;
totalThetaRight += diff;
```

Für die Navigation wurde ein kontinuierlicher, nicht blockierender Fahrmodus ergänzt (Kommando R:). Während der Fahrt gleicht der Teensy die beiden Radwege selbstständig ab: Läuft das rechte Rad weiter als das linke, wird proportional gegengesteuert. Diese Geradeauskorrektur läuft mit der Odometrie-Abtastung alle 50 ms direkt auf dem Mikrocontroller und damit deutlich schneller, als es der Pi über Funk-Umwege könnte:

```
double delta = fabs(totalThetaRight) - fabs(totalThetaLeft);
int auto_corr = constrain((int)(delta * 3.0), -40, 40);
int pwm_r = constrain(base_right + total_corr, 1400, 1490);
int pwm_l = constrain(base_left  + total_corr, 1560, 1630);
```

Drehungen auf der Stelle (T:) fahren beide Räder gegenläufig, bis die Summe der Radbögen der Ziel-Bogenlänge auf dem Drehkreis (Durchmesser 0,206 m, Korrekturfaktor 1,03 gegen Überschwingen) entspricht. Entscheidend für den Filter: Der Teensy meldet nach jedem Kommando nicht den Sollwert, sondern den aus der Odometrie berechneten Ist-Wert zurück (T:<ist_grad> bzw. D:<ist_m>). Der Pi füttert den Partikelfilter also mit dem, was wirklich gefahren wurde, nicht mit dem, was bestellt war. Beim Anhalten aus voller Fahrt wird die Geschwindigkeit in zehn Stufen abgebaut, damit der Roboter nicht nickt und die Odometrie beim Ausrollen gültig bleibt.

Navigator: Zustandsmaschine und MCL-Zyklus

Die Node navigator.py setzt die Waypoint-Routine als Zustandsmaschine um, die mit 10 Hz getaktet ist. Der Grundzyklus für ein Segment lautet: Ausrichten (ALIGN), segmentweise fahren (DRIVING), am Waypoint anhalten und final messen (STOP_UPDATE), Pose prüfen (CHECK_POSE), gegebenenfalls korrigieren (CORRECT), weiter zum nächsten Punkt (NEXT). Gefahren wird nicht in einem Stück, sondern in Segmenten von 0,30 m: Nach jedem Segment liefert der Teensy sein D:-Feedback, und genau dann läuft ein kompletter MCL-Zyklus (motion_predict mit der Ist-Distanz, Messupdate mit dem aktuellen Scan, Resampling, Pose-Schätzung). Das entspricht der Empfehlung aus der Vorlesung, alle 10 bis 20 cm oder an Stopps zu aktualisieren, angepasst an die tatsächliche Fahrdynamik des Roboters.

Am Waypoint selbst wird im Stillstand ein zusätzliches Messupdate gefahren, weil der Scan ohne Bewegungsunschärfe die beste Messung des ganzen Zyklus ist. Danach entscheidet CHECK_POSE anhand der Bewertungslogik der Aufgabenstellung: Abweichung bis 3 cm gilt als voll erreicht, bis 8 cm wird eine Korrekturbewegung versucht, darüber wird bis zu dreimal korrigiert, bevor der Punkt aufgegeben wird:

```
err = math.hypot(wp['x'] - self.pose[0], wp['y'] - self.pose[1])
if err <= tol_full:      # 0.03 m -> volle Punktzahl
```

```
        self.state = State.NEXT
    elif err <= tol_half:    # 0.08 m -> Korrektur versuchen
        self.state = State.CORRECT
```

Beim Übergang zum nächsten Punkt wird die Partikelwolke eng auf den soeben bestätigten Waypoint re-initialisiert. Der bestätigte Punkt ist die verlässlichste Information im System, und die Re-Initialisierung verhindert, dass sich ein Restfehler der Schätzung über mehrere Waypoints aufschaukelt. Jeder Streckenabschnitt startet so mit einer sauberen, eng lokalisierten Wolke.

Lidar-gestützte Feinnavigation und Sonderfälle

Die MCL-Schätzung liefert die globale Pose, für das zentimetergenaue Anhalten nutzt der Navigator zusätzlich direkte Lidar-Referenzen auf die aus den Referenzscans bekannten Wände. Dafür gibt es Hilfsfunktionen, die aus dem Scan den minimalen Abstand nach vorn (Kegel einstellbar), nach hinten (um 180 Grad) und zur jeweils linken Wand (um +90 Grad) extrahieren. Ein Waypoint, der z. B. 0,35 m vor einer bekannten Wand liegt, wird über einen Lidar-Frontstopp angefahren: Der Roboter fährt Segmente, bis die Frontdistanz den konfigurierten Zielwert unterschreitet, und stoppt dann sofort per S:-Kommando. Das ist unabhängig von jedem Odometrie-Drift und wiederholgenau.

Auf langen Geraden hält ein Wandfolge-Regler den Sollabstand zur Seitenwand (z. B. 0,26 m auf P1 nach P2). Er arbeitet bewusst nicht kontinuierlich, sondern korrigiert zwischen zwei Segmenten mit einer kleinen Drehung: Aus dem Abstandsfehler wird über atan2(fehler, segmentlaenge) ein Zielwinkel berechnet, gedämpft, auf wenige Grad begrenzt und gegen das IMU-Heading seit Segmentstart verrechnet. Meldet die IMU mehr als 15 Grad Abweichung, wird der Messwert als Drift verworfen und nur die Referenz neu gesetzt, so kann ein IMU-Fehler nie eine große Falschdrehung auslösen:

```
dist_err = min_dist - self._wall_target
target_deg = math.degrees(math.atan2(dist_err, seg)) * 0.5
target_deg = max(-5.0, min(5.0, target_deg))
turn_deg = target_deg - heading_deg
if abs(turn_deg) > 1.0:
    self._send_cmd(f'T:{turn_deg:.1f}')
```

Einige Streckenabschnitte brauchen eigene Sequenzen, die als zusätzliche Zustände implementiert sind. Drei Beispiele: Der Weg zu P3 ist zweiphasig (erst nach Süden bis zum Frontstopp an der unteren Wand, dort 90 Grad drehen, dann nach Westen mit Wandfolge), wobei der Drehwinkel nicht blind 90 Grad beträgt, sondern um den per Lidar gemessenen Winkel der Vorderwand korrigiert wird. Bei P6 zeigte sich, dass der Scan direkt nach einer 90-Grad-Drehung kurzzeitig zu kurze Distanzen liefert (gemessen 1,42 m bei real 1,98 m); der Frontstopp wird dort deshalb nur an Segmentgrenzen und mit sehr engem 2-Grad-Kegel geprüft statt in jedem Takt. Auf dem Weg zu P7 wächst der Abstand zur Rückwand statt

dass die Front schrumpft, also stoppt dieser Abschnitt auf die Rückwanddistanz. Vor der 180-Grad-Drehung bei P8 wird das Heading zunächst per IMU auf den Wert vom Beginn der Westfahrt zurückkorrigiert (bis zu drei Versuche, Schwelle 2 Grad), damit sich die kleinen Pendelkorrekturen der Wandfolge nicht in die Drehung fortpflanzen.

Sicherheit und Logging

Bleibt nach einem Kommando das Feedback aus (z. B. Teensy-Agent noch nicht verbunden), sendet der Navigator nach 5 s Timeout erneut, statt ewig zu warten. Der Not-Stopp über den Wert 500 auf dem alten Topic funktioniert jederzeit. Jeder Lauf schreibt ein CSV-Log (Zeit, Zustand, Waypoint, Pose-Schätzung, N_eff) und speichert regelmäßig komplette Partikel-Snapshots als Pickle-Datei, sodass sich der Filter nach dem Lauf Schritt für Schritt nachvollziehen und visualisieren lässt.

Gesamtsystem

Insgesamt bildet das Projekt einen geschlossenen Kreis aus Wahrnehmung, Schätzung und Bewegung: Der Teensy fährt Segmente und meldet Ist-Odometrie, der Partikelfilter verrechnet diese Bewegung mit dem Lidar-Abgleich gegen die vermessene Karte, und die Zustandsmaschine übersetzt die Schätzung in das nächste Kommando. Die wichtigsten Stellschrauben sind die Segmentlänge (0,30 m), die empirisch gemessenen Rauschparameter des Bewegungsmodells, die Lidar-Streuung sigma = 1 cm mit Likelihood-Floor 0,01, die Strahlanzahl 36, die Sollabstände und Stopp-Distanzen der Wandreferenzen sowie die Begrenzungen der Korrekturdrehungen. Bewusste Entwurfsentscheidung ist die Arbeitsteilung der Sensoren: Die Odometrie treibt die Bewegungsvorhersage, der Lidar verankert die Schätzung in der Karte und übernimmt das präzise Anhalten an den Wänden, die IMU stabilisiert kurzfristig das Heading, wird wegen ihres Yaw-Drifts aber nie als absolute Referenz verwendet.